Recursion, Tree Recursion Meta

Computer Science Mentors 61A

September 21, 2026 - September 25, 2026

Example Timeline

- Intros & Logistics [5 min]
- Recursion Basics: base cases, recursive cases, and the recursive leap of faith [10 min]
- Writing Recursive Functions: turning a description of a process into code [15 min]
- Recursive Helper Functions: carrying extra state through a helper [12 min]
- Tree Recursion: exploring multiple choices with multiple recursive calls [18 min]

  Don't

1 Recursion Basics

1. What are three parts you find in every recursive function?

   1. One or more base case(s)
   2. One or more ways to break the problem down into a smaller problem
   3. Solve the smaller problem(s) recursively and combine the results to solve the original problem

   Suggested Time: 3 min; Difficulty: Easy

   - Feel free to give an example of a recursive function when explaining this question, and keep getting students to identify parts of recursive functions while solving problems!

Created by Ana Abell, Tien-Yu Hsu, Cadence Loh, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

2. What is wrong with the following function? How can we fix it?

```
def factorial(n):
    return n * factorial(n)
```

There is no base case and the recursive call is made on the same `n`.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
```

This is a good opportunity to emphasize to your students that the return type of the base case should match the return type of the function (e.g., if the function is expected to return an integer, the base case must also return an integer).

Suggested Time: 5 min; Difficulty: Easy

- Ask students to trace `factorial(3)` to see why it never terminates before asking them to fix it.
- Emphasize that the base case's return type should match the type the rest of the function returns.

3. Complete the definition for `sum_prime_digits`, which returns the sum of all the prime digits of `n`. Recall that 1 is not prime. Assume you have access to a function `is_prime`; `is_prime(n)` returns `True` if `n` is prime, and `False` otherwise.

```
def sum_prime_digits(n):
    """
    >>> sum_prime_digits(12345)
    10 # 2 + 3 + 5
    >>> sum_prime_digits(4681029)
    2 # 2 is the only prime number
    """
    if _____________________________________________:

        return _____________________________________

    if _____________________________________________:

        return _____________________________________

    return _____________________________________
```

```
    if n == 0:
        return 0
    if is_prime(n % 10):
        return n % 10 + sum_prime_digits(n // 10)
    return sum_prime_digits(n // 10)
```

- Again, this problem is likely skippable if you have advanced students.

- Make sure your students understand why you are calling `sum_prime_digits` twice.

Suggested Time: 5 min; Difficulty (Official): Medium; Difficulty (Adjusted): Easy → Medium Mezzanine

2 Writing Recursive Functions

1. Complete the definition for `num_digits`, which takes in a number n and returns the number of digits it has . Implement the function recursively.

```
def num_digits(n):
    """Takes in an positive integer and returns the number of
    digits.

    >>> num_digits(0)
    1
    >>> num_digits(1)
    1
    >>> num_digits(7)
    1
    >>> num_digits(1093)
    4
    """


    if n < 10:
        return 1
    else:
        return 1 + num_digits(n // 10)
```

Suggested Time: 5 min; Difficulty: Easy

- Good warm-up for describing the recursive case in English first ("the number of digits of n is 1 more than the number of digits of n without its last digit") before writing code.

2. (Spring 2015 MT1 Q3C) Implement the `combine` function, which takes a non-negative integer `n`, a two-argument function `f`, and a number `result`. It applies `f` to the first digit of `n` and the result of combining the rest of the digits of `n` by repeatedly applying `f` (see the doctests). If `n` has no digits (because it is zero), `combine` returns `result`.

```
def combine(n, f, result):
    """
    Combine the digits in non-negative integer n using f.

    >>> combine(3, mul, 2) # mul(3, 2)
    6
    >>> combine(43, mul, 2) # mul(4, mul(3, 2))
    24
    >>> combine(6502, add, 3) # add(6, add(5, add(0, add(2, 3))))
    16
    >>> combine(239, pow, 0) # pow(2, pow(3, pow(9, 0)))
    8
    """
    if n == 0:
        return result
    else:
        return combine(_______ , _______ ,

                       _____________________________)
```

```
def combine(n, f, result):
    if n == 0:
        return result
    else:
        return combine(n // 10, f, f(n % 10, result))
```

Teaching Tips

- Visually go through an example (like using one from the doc tests) and going over the question visually is especially helpful when showing students how to make the original problem into a smaller subproblem.
  - Ex. combine(43, mul, 2) # mul(4, mul(3, 2))
- Common misconceptions(points to emphasize)
  - The order of arguments for f matters (n % 10, result)
  - The third parameter is evaluated before the combine function is applied to the parameters.

Suggested Time: 10 min; Difficulty: Medium

- Walk through `combine(43, mul, 2)` concretely before filling in blanks, the same way lecture worked through `streak` by picking an example and describing the process in words first.
- Common misconception: the order of arguments to `f` matters (`f(n % 10, result)`, not the reverse).

3 Recursive Helper Functions

1. Suppose the **position** of the rightmost digit of a number is defined as 1. The position of a digit increases from right to left.

   (a) Fill in the `positionizer` function below. It takes in a non-negative integer `n` and returns a non-negative integer. For each digit `d` of `n`, `positionizer` either changes `d` to be the remainder of `d` divided by its position, or leaves `d` the same if it is equal to its position.

```
def positionizer(n):
    """
    >>> positionizer(12)
    10
    >>> positionizer(23)
    20
    >>> positionizer(12345)
    12300
    """
    def helper(n, pos):

        if _____________________________________________:

            return _____________________________________

        rest = _____________________________________
        if n % 10 == pos:

            return rest + _____________________________

        else:

            return rest + _____________________________

    return helper(_________________, _________________)
```

```
def positionizer(n):
    def helper(n, pos):
        if n == 0:
            return 0
        rest = helper(n // 10, pos + 1) * 10
        if n % 10 == pos:
            return rest + n % 10
        else:
            return rest + (n % 10) % pos
    return helper(n, 1)
```

Teaching Tips

- Remind students we define the rightmost digit to be at position 1, increasing by 1 as you traverse left.
- An important question to consider is why we need to create a helper function in the first place? What additional information does the helper allow us to store?

- What operators do we use to get the last digit? The rest of the number?
- Emphasize the recursive leap of faith in calling `helper(n // 10, pos + 1)` to give us the correct result for the rest of `n`.

(b) Now fill in the `max_positionizer` function below. Given a list of digits, it returns the `k`-digit number with the highest value when positionized. The digits must stay in the same order. (ex: for `k = 2` and `lst = [1,2,3]`, you can't form 21). You may use `positionizer` in your solution.

Hint: Use the helper to create a list of all possible k digit numbers formed from `lst`.

```
def max_positionizer(k, lst):
  """
  >>> max_positionizer(2, [1, 2, 3]) # positionized version
                                     # of 12, 13, 23 are
                                     # 10, 10, 20
  23
  >>> max_positionizer(3, [2, 5, 3, 1])
  251
  """
  def make_nums(k,lst):

    if ____________________________________:

      return ____________________________________

    elif ____________________________________:
      return []

    a =  [_____________________________ \

              for rest in _____________________________]

    b = ____________________________________________
    return a + b

  return ________(make_nums(_____, _____), _____________)
```

```
def max_positionizer(k, lst):
  def make_nums(k,lst):
    if k == 0: # Note that the check for k must come first
               # (what should be returned if k == 0 and
               # len(lst) == 0?)
      return [0]
    elif len(lst) == 0:
      return []
    a =  [lst[0] * 10**(k - 1) + rest \
            for rest in make_nums(k - 1, lst[1:])]
    b = make_nums(k, lst[1:])
    return a + b
  return max(make_nums(k, lst), key=positionizer)
```

Teaching Tips

- Remind students about the key parameter of the max function and how it works
- How do we create all orderings of numbers of `k` digits in `a + b`?

- As always, think about all possible base cases first: the base case for "k" is when it is 0, and the base case for "lst" is when it is empty.

Suggested Time: 10 min (Part a); Part b is a stretch problem, do it only if time allows.

- The given function doesn't have enough information on its own, so we introduce a helper function that takes an extra argument (`pos`) to track additional state as we recurse.
- Ask: "What does `helper` need to know that `positionizer` doesn't pass it directly?" and "What should we call `helper` with initially?".

4 Tree Recursion

1. What is the difference between recursion and tree recursion? Give an example of a problem that might require tree recursion.

   In the recursive functions we've seen so far, each recursive call makes only one call to itself. In tree recursion, a single function call can make multiple recursive calls, causing the calls to branch out like the branches of a tree (rather than a single chain).

   Despite this difference, tree recursive problems are solved the exact same way as any other recursive problem: find the base case(s), take the recursive leap of faith by trusting that each of our recursive calls correctly solves its subproblem, and combine the results of our subproblems to solve the original problem. The only real difference is that we may need to combine *more than one* subproblem's result to get our answer.

   Tree recursion often comes up in counting problems (how many ways are there to do something?) and optimization problems (what is the max/min number of ways to do something?). For example, counting the number of ways to make change for an amount using a set of coin denominations, or computing the nth Fibonacci number, both require branching into multiple recursive calls per frame.

   Because tree recursive functions have more moving pieces, it can help to break the design process into three questions.

   - Base case(s): What are the simplest possible arguments to the function? (Doctests are often a good hint here.)
   - Recursive calls: What options should be recursively explored at each step? (Drawing a tree diagram of the recursive calls can help visualize this.)
   - Combining: How should the results of these subproblems be combined? (Trust the recursive leap of faith, then combine the results with a mathematical or logical operator.)

   Even though a tree recursive function may branch into many, many recursive calls, we only need to reason about one level of the tree at a time – the recursive leap of faith handles the rest.

   Suggested time: 3 min

2. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the nth Gibonacci number g_n. The first three terms of the sequence are $g_0 = 0, g_1 = 1, g_2 = 2$. For $n \geq 3$, g_n is defined as the sum of the previous three terms in the sequence.

   Complete the function `gib`, which takes in an integer `n` and returns the nth Gibonacci number, g_n. Also, identify the three parts of recursive function design as they are used in your solution.

```
def gib(n):
    """
    >>> gib(0)
    0
    >>> gib(1)
    1
    >>> gib(2)
    2
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3
    3
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6
    6
    """
    if ____________________________________:

        return ____________________________________

    return ____________________________________
```

```
def gib(n):
    if n <= 2:
        return n
    return gib(n - 1) + gib(n - 2) + gib(n - 3)
```

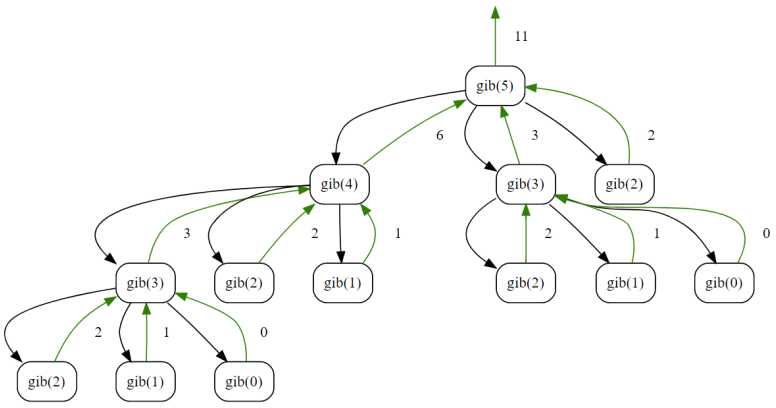
- Base case: `if n <= 2: return n`
- Recursive calls: `gib(n - 1), gib(n - 2), gib(n - 3)`
- Solving the larger problem: adding the results of the three recursive calls.

- This problem is very similar to Fibonacci, which students should be familiar with from lecture.
- Ensure that students understand how the minor alteration to the sequence slightly modifies the recursive calls and the base case

3. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

   13.

   There are a few ways you could determine this. First, you could draw out a call graph that shows all of the calls that are made. `https://tinyurl.com/gibsol`



   You could also build things from the ground up using recursive intuition.

   - `gib(0)` takes one call because it's a base case.
   - `gib(1)` takes one call because it's a base case.
   - `gib(2)` takes one call because it's a base case.
   - `gib(3)` takes four calls: one for the original call, one for `gib(0)`, one for `gib(1)`, and one for `gib(2)`.
   - `gib(4)` takes seven calls: one for the original call, four for `gib(3)`, one for `gib(2)`, and one for `gib(1)`.
   - `gib(5)` takes 13 calls: one for the original call, seven for `gib(4)`, four for `gib(3)`, and one for `gib(2)`.

   This second method is much faster, especially for higher values of `n`.

   In general, I believe that drawing out call graphs can be confusing for students, especially for more difficult recursive problems. The whole idea behind recursion, after all, is that you don't have to worry about the dense tree of calls that result from a single call to a recursive function—you can just trust that the recursive calls you do make return the correct result and don't have to worry about how they specifically got that result. However, I believe that it is useful to do this sort of thing once because it demonstrates how recursion actually works in practice and why the recursion is valid and useful.

   When teaching this problem, I recommend first going over the call tree. I would emphasize a few ideas. First, our base cases are at the bottom of the call graph, and every call eventually terminates at one of the base cases. They literally serve as the foundation of the tree. They are the "truth" upon which the function's validity for higher inputs rests. I would also point out that every call that's not a base case has three calls underneath it because each of these calls makes three recursive calls. Hopefully this makes the idea of recursion more concrete for the students and qualms perennial concerns that the logic behind recursion is circular.

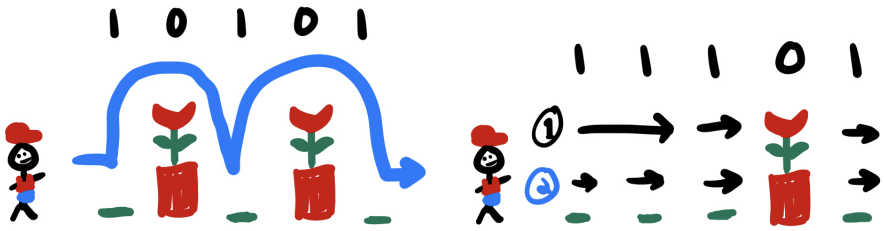
   I would then going over the "smarter" way of doing the problem. I would point out how doing the problem in this way allows us to avoid the repetitive, confusing, and error prone work of considering

every call in the function. I would note that if we tried to do the same thing with `gib(1000)`, the graph would be untenable. The lesson I want them to walk away with is: abstracting away the huge number of calls is a valuable tool when working with recursion, and that they should try to avoid thinking about these calls if at all possible, as it only gets more confusing for more complicated functions.

Suggested Time: 5 min; Difficulty: Easy

- This is structurally the same idea as lecture's `count_partitions`: at each step we explore using vs. not using a given coin size, except here we take a **`min`** over choices instead of summing them.
- Ask students what changes if the question instead asked for the *number of ways* to make change rather than the minimum number of coins – this connects directly back to `count_partitions`.

4. Mario needs to get from one end of a level to the other, but there are deadly Piranha plants in his way! Mario only moves forward and can either *step* (move forward one space) or *jump* (move forward two spaces) from each position. A level is represented as a series of ones and zeros, with zeros denoting the location of Piranha plants. Mario can step on ones but not on zeros. How many different ways can Mario traverse a level without stepping or jumping into a Piranha plant? Assume that every level begins with a 1 (where Mario starts) and ends with a 1 (where Mario must end up).



Hint: Does it matter whether Mario goes from left to right or right to left? Which one is easier to check?

```
def mario_number(level):
    """
    >>> mario_number(10101)
    1
    >>> mario_number(11101)
    2
    >>> mario_number(100101)
    0
    """
    if _____________________________:

        _____________________________

    elif _____________________________:

        _____________________________

    else:

        ___________________________________________________________
```

```
def mario_number(level):
    if level == 1:
        return 1
    elif level % 10 == 0:
        return 0
    else:
        return mario_number(level // 10) + mario_number((level // 10) //
            10)
```

You can think about this tree recursion problem as testing out all of the possible ways Mario can traverse the level, and adding 1 every time you find a possible traversal.
Here it doesn't matter whether Mario goes left to right or right to left; either way we'll end up with the same number of ways to traverse the `level`. In that case, we can simply choose for Mario to start from the right, and then we can process the level like we process other numbers in digit-parsing related questions by using floor division (`//`) and modulo (`%`)
At each point in time, Mario can either step or jump. We use a single floor division (`//`) of `level` by 10 to represent taking one step (if we took a step, then the entire `level` would be left except for the last number), while two floor divisions by 10 (or equivalently one floor division by 100) corresponds to a jump at this point in the `level` (if we took a jump, then the entire `level` would be left except for the last two numbers).
To think of the base cases, you can consider the two ways that Mario ends his journey. The first, corresponding to `level == 1`, means that Mario has successfully reached the end of the level. You can `return 1` here, because this means you've found one additional path to the end. The second, corresponding to `level % 10 == 0`, means that Mario has landed on a Piranha plant. This returns 0 because it's a failed traversal of the `level`, so you don't want to add anything to your result.
In tree recursion, you need to find a way to combine separate recursive calls. In this case, because `mario_number` returns an integer and the base cases are integers and you're trying to count the total number of ways of traversal, it makes sense to add your recursive calls.

Teaching Tips

- Some leading questions:
  - What are our base cases? (When do we know we've reached the end of the level? When do we know that we've failed?)
  - Is there any difference between going left to right or right to left in terms of the number of ways to traverse the level?
  - What are our two options at each step?
  - What do those look like in a recursive call?
  - How should we combine our recursive calls? (and, or, addition, etc.)
- Try leaning into the narrative of the question! It's fun and can help rephrase recursive calls "in plain english" I also like drawing the problem out along with the doctests to visualize the different steps Mario can take :D !
- It's very useful to draw a tree diagram! Each function call has one branch for stepping once and another branch for stepping twice. Each branch then has 2 branches of their own (until a base case is reached).
- Teach students that recursive calls can be treated as numbers using the recursive leap of faith, so combining the two recursive call branches with addition is really just adding two numbers.

Suggested Time: 10 min; Difficulty (Official): Medium; The classic mario-number problem from CSM. A worksheet wouldn't be complete without it!

5. Implement the function `make_change`, which takes in a non-negative integer amount in cents `n` and returns the minimum number of coins needed to make change for `n` using 1-cent, 3-cent, and 4-cent coins.

```
def make_change(n):
    """
    >>> make_change(5) # 5 = 4 + 1 (not 3 + 1 + 1)
    2
    >>> make_change(6) # 6 = 3 + 3 (not 4 + 1 + 1)
    2
    """

    if _____________________:
        return 0

    elif _____________________:

        ________________________________________

    elif _____________________:

        ________________________________________
    else:

        ________________________________________


def make_change(n):
    if n == 0:
        return 0
    elif n < 3:
        return 1 + make_change(n - 1)
    elif n < 4:
        return 1 + min(make_change(n - 1), make_change(n - 3))
    else:
        return 1 + min(make_change(n - 1), make_change(n - 3),
            make_change(n - 4))
```

Teaching Tips

- Be careful of the wording here - we're counting the **minimum number of coins** used to make change, not the number of ways to make change.
- Go over the doctests to show that choosing the largest coin at the beginning isn't always the most efficient; we have to explore all possibilities.
- Some questions you can ask your students:
  - What are the different options available to us for making change at each step?
  - What are some cases where not all coins are available to us?
  - In the case that there are multiple options available, which one do we pick?
    - Is there a Python function that we can use to choose the right combination?
- Note that this skeleton is directed towards leading students to the answer (by building up from only using one coin, to using two, to using three). It is not the most efficient one (for example, you can just return n if $n < 3$).
- If your students come up with a more compact solution, use that as an opportunity to solicit ideas from your entire section about how to reduce the complexity! You don't always have to present the exact solution.

Suggested Time: 10 min; Difficulty: Medium

- This is structurally the same idea as lecture's `count_partitions`: at each step we explore using vs. not using a given coin size, except here we take a **`min`** over choices instead of summing them.
- Ask students what changes if the question instead asked for the *number of ways* to make change rather than the minimum number of coins – this connects directly back to `count_partitions`.